

LayoutDiffusion: Controllable Diffusion Model for Layout-to-image Generation

Guangcong Zheng^{1*}, Xianpan Zhou^{2*}, Xuewei Li^{1†}, Zhongang Qi³, Ying Shan³, Xi Li^{1,4,5,6†}

¹College of Computer Science & Technology, Zhejiang University

²Polytechnic Institute, Zhejiang University ³ARC Lab, Tencent PCG

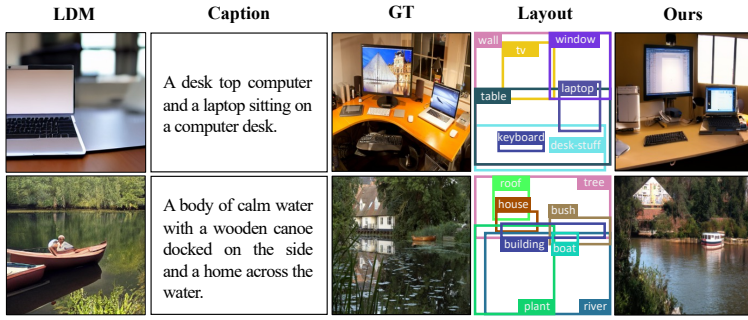
⁴Shanghai Institute for Advanced Study of Zhejiang University

⁵Shanghai AI Lab ⁶Zhejiang – Singapore Innovation and AI Joint Research Lab

{guangcongzheng, zhouxianpan, xueweili, xilizju}@zju.edu.cn

{zhongangqi, yingshan}@tencent.com

(a) layout-guided vs. text-guided image generation for diffusion model



(b) Image-Layout Fusion in a unified space

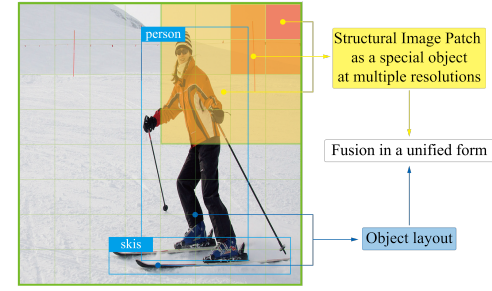


Figure 1. Compared to text, the layout allows diffusion models to obtain more control over the objects while maintaining high quality. Unlike the prevailing methods, we propose a diffusion model named LayoutDiffusion for layout-to-image generation. We transform the difficult multimodal fusion of the image and layout into a unified form by constructing a structural image patch with region information and regarding the patched image as a special layout.

Abstract

Recently, diffusion models have achieved great success in image synthesis. However, when it comes to the layout-to-image generation where an image often has a complex scene of multiple objects, how to make strong control over both the global layout map and each detailed object remains a challenging task. In this paper, we propose a diffusion model named LayoutDiffusion that can obtain higher generation quality and greater controllability than the previous works. To overcome the difficult multimodal fusion of image and layout, we propose to construct a structural image patch with region information and transform the patched image into a special layout to fuse with the normal layout in a unified form. Moreover, Layout Fusion Module (LFM) and Object-aware Cross Attention (OaCA) are proposed to model the relationship among multiple objects and designed to be object-aware and position-sensitive, allow-

ing for precisely controlling the spatial related information. Extensive experiments show that our LayoutDiffusion outperforms the previous SOTA methods on FID, CAS by relatively 46.35%, 26.70% on COCO-stuff and 44.29%, 41.82% on VG. Code is available at <https://github.com/ZGTroy/LayoutDiffusion>.

1. Introduction

Recently, the diffusion model has achieved encouraging progress in conditional image generation, especially in text-to-image generation such as GLIDE [25], Imagen [33], and Stable Diffusion [32]. However, text-guided diffusion models may still fail in the following situations. As shown in Fig. 1 (a), when aiming to generate a complex image with multiple objects, it is hard to design a prompt properly and comprehensively. Even input with well-designed prompts, problems such as missing objects and incorrectly generating objects' positions, shapes, and categories still occur in the state-of-the-art text-guided diffusion model [25, 32, 33].

*Equal contribution.

†Corresponding author.

This is mainly due to the ambiguity of the text and its weakness in precisely expressing the position of the image space. Fortunately, this is not a problem when using the coarse layout as guidance, which is a set of objects with the annotation of the bounding box (bbox) and object category. With both spatial and high-level semantic information, the diffusion model can obtain more powerful controllability while maintaining the high quality.

However, early studies [2, 14, 41, 45] on layout-to-image generation are almost limited to generative adversarial networks (GANs) and often suffer from unstable convergence [1] and mode collapse [28]. Despite the advantages of diffusion models in easy training [10] and significant quality improvement [7], few studies have considered applying diffusion in the layout-to-image generation task. To our knowledge, only LDM [32] supports the condition of layout and has shown encouraging progress in this field.

In this paper, different from LDM that applies the simple multimodal fusion method (e.g., the cross attention) or direct input concatenation for all conditional input, we aim to specifically design the fusion mechanism between layout and image. Moreover, instead of conditioning only in the second stage like LDM, we propose an end-to-end one-stage model that considers the condition for the whole process, which may have the potential to help mitigate loss in the task that requires fine-grained accuracy in pixel space [32]. The fusion between image and layout is a difficult multimodal fusion problem. Compared to the fusion of text and image, the layout has more restrictions on the position, size, and category of objects. This requires a higher controllability of the model and often leads to a decrease in the naturalness and diversity of the generated image. Furthermore, the layout is more sensitive to each token and the loss in token of layout will directly lead to the missing objects.

To address the problems mentioned above, we propose treating the patched image and the input layout in a unified form. Specifically, we construct a structural image patch at multi-resolution by adding the concept of region that contains information of position and size. As a result, each patch of the image is transformed into a special type of object, and the entire patched image will also be regarded as a layout. Finally, the difficult problem of multimodal fusion between image and layout will be transformed into a simple fusion with a unified form in the same spatial space of the image. We name our model LayoutDiffusion, a layout-conditional diffusion model with Layout Fusion Module (LFM), object-aware Cross Attention Mechanism (OaCA), and corresponding classifier-free training and sampling scheme. In detail, LFM fuses the information of each object and models the relationship among multiple objects, providing a latent representation of the entire layout. To make the model pay more attention to the information re-

lated to the object, we propose an object-aware fusion module named OaCA. Cross-attention is made between the image patch feature and layout in a unified coordinate space by representing the positions of both of them as bounding boxes. To further improve the user experience of LayoutDiffusion, we also make several optimizations on the speed of the classifier-free sampling process and could significantly outperform the SOTA models in 25 iterations.

Experiments are conducted on COCO-stuff [5] and Visual Genome (VG) [19]. Various metrics ranging from quality, diversity, and controllability show that LayoutDiffusion significantly outperforms both state-of-the-art GAN-based and diffusion-based methods.

Our main contribution is listed below.

- Instead of using the dominated GAN-based methods, we propose a diffusion model named LayoutDiffusion for layout-to-image generations, which can generate images with both high-quality and diversity while maintaining precise control over the position and size of multiple objects.
- We propose to treat each patch of the image as a special object and accomplish the difficult multimodal fusion of layout and image in a unified form. LFM and OaCA are then proposed to fuse the multi-resolution image patches with user’s input layout.
- LayoutDiffusion outperforms the SOTA layout-to-image generation method on FID, DS, CAS by relatively around 46.35%, 9.61%, 26.70% on COCO-stuff and 44.29%, 11.30%, 41.82% on VG.

2. Related work

The related works are mainly from layout-to-image generation and diffusion models.

Layout-to-Image Generation. Before the layout-to-image generation is formally proposed, the layout is usually used as a complementary feature [15, 31, 43] or an intermediate representation in text-to-image [12], scene-to-image generation [14]. The first image generation directly from the layout appears in Layout2Im [50] and is defined as a set of objects annotated with category and bbox. Models that work well with fine-grained semantic maps at the pixel level can also be easily transformed to this setting [13, 27, 46]. Inspired by StyleGAN [16], LostGAN-v1 [40], LostGAN-v2 [41] used a reconfigurable layout to obtain better control over individual objects. For interactive image synthesis, PLGAN [45] employed panoptic theory [18] by constructing stuff and instance layouts into separate branches and proposed Instance- and Stuff-Aware Normalization to fuse into panoptic layouts. Despite encouraging progress in this field, almost all approaches are limited to the generative

adversarial network (GAN) and may suffer from unstable convergence [1] and mode collapse [28]. As a multimodal diffusion model, LDM [32] supports the condition of coarse layout and has shown great potential in layout-guided image generation.

Diffusion Model. Diffusion models [3, 10, 26, 32, 36, 38, 39, 47] are being recognized as a promising family of generative models that have proven to be state-of-the-art sample quality for a variety of image generation benchmarks [6, 44, 48], including class-conditional image generation [7, 51], text-to-image generation [25, 32, 33], and image-to-image translation [17, 23, 34]. Classifier guidance was introduced in ADM-G [7] to allow diffusion models to condition the class label. The gradient of the classifier trained on noised images could be added to the image during the sampling process. Then Ho et al. [11] proposed a classifier-free training and sampling strategy by interpolating between predictions of a diffusion model with and without condition input. For the acceleration of training and sampling speed, LDM proposed to first compress the image into smaller resolution and then apply denoising training in the latent space.

3. Method

In this section, we propose our LayoutDiffusion, as shown in Fig. 2. The whole framework consists mainly of four parts: (a) layout embedding that preprocesses the layout input, (b) layout fusion module that encourages more interaction between objects of layout, (c) image-layout fusion module that constructs the structural image patch and object-aware cross attention developed with the specific design for layout and image fusion, (d) the layout-conditional diffusion model with training and accelerated sampling methods.

3.1. Layout Embedding

A layout $l = \{o_1, o_2, \dots, o_n\}$ is a set of n objects. Each object o_i is represented as $o_i = \{b_i, c_i\}$, where $b_i = (x_0^i, y_0^i, x_1^i, y_1^i) \in [0, 1]^4$ denotes a bounding box (bbox) and $c_i \in [0, C + 1]$ is its category id.

To support the input of a variable length sequence, we need to pad l to a fixed length k by adding one o_l in the front and some padding o_p in the end, where o_l represents the entire layout and o_p represents no object. Specifically, $b_l = (0, 0, 1, 1)$, $c_l = 0$ denotes a object that covers the whole image and $b_p = (0, 0, 0, 0)$, $c_p = C + 1$ denotes a empty object that has no shape or does not appear in the image.

After the padding process, we can get a padded $l = \{o_1, o_2, \dots, o_k\}$ consisting of k objects, and each object has its specific position, size, and category. Then, the layout l is transformed into a layout embedding $L = \{O_1, O_2, \dots, O_k\} \in \mathbb{R}^{k \times d_L}$ by the projection matrix $W_B \in$

$\mathbb{R}^{4 \times d_L}$ and $W_C \in \mathbb{R}^{1 \times d_L}$ using the following equation:

$$L = B_L + C_L \quad (1)$$

$$B_L = bW_B \quad (2)$$

$$C_L = cW_C \quad (3)$$

where $B_L, C_L \in \mathbb{R}^{k \times d_L}$ are the bounding box embedding and the category embedding of a layout l , respectively. As a result, L is defined as the sum of B_L and C_L to include both the content and positional information of a entire layout, and d_L is the dimension of the layout embedding.

3.2. Layout Fusion Module

Currently, each object in layout has no relationship with other objects. This leads to a low understanding of the whole scene, especially when multiple objects overlap and block each other. Therefore, to encourage more interaction between multiple objects of the layout to better understand the entire layout before inputting the layout embedding, we propose Layout Fusion Module (LFM), a transformer encoder that uses multiple layers of self-attention to fuse the layout embedding and can be denoted as

$$L' = \text{LFM}(L) \quad (4)$$

, where the output is a fused layout embedding $L' = \{O'_1, O'_2, \dots, O'_k\} \in \mathbb{R}^{k \times d_L}$.

3.3. Image-Layout Fusion Module

Structural Image Patch. The fusion of image and layout is a difficult multimodal fusion problem, and one of the most important parts lies in the fusion of position and size. However, the image patch is limited to the semantic information of the whole feature and lacks the spatial information. Therefore, we construct a structural image patch by adding the concept of region that contains the information of position and size.

Specifically, $I \in \mathbb{R}^{h \times w \times d_I}$ denotes the feature map of a entire image with height h , width w , and channel d_I . We define that $I_{u,v}$ is the u^{th} row and v^{th} column patch of I and its bounding box, or the ablated region information, is defined as $b_{I_{u,v}}$ by the following equation:

$$b_{I_{u,v}} = \left(\frac{u}{h}, \frac{v}{w}, \frac{u+1}{h}, \frac{v+1}{w} \right) \quad (5)$$

The bounding box sets of a patched image I is defined as $b_I = \{b_{I_{u,v}} | u \in [0, h), v \in [0, w)\}$. As a result, the positional information of image patch and layout object is contained in the unified bounding box defined in the same spatial space, leading to better fusion of image and layout.

Positional Embedding in Unified Space. We define the positional embedding of the image and layout as P_I and

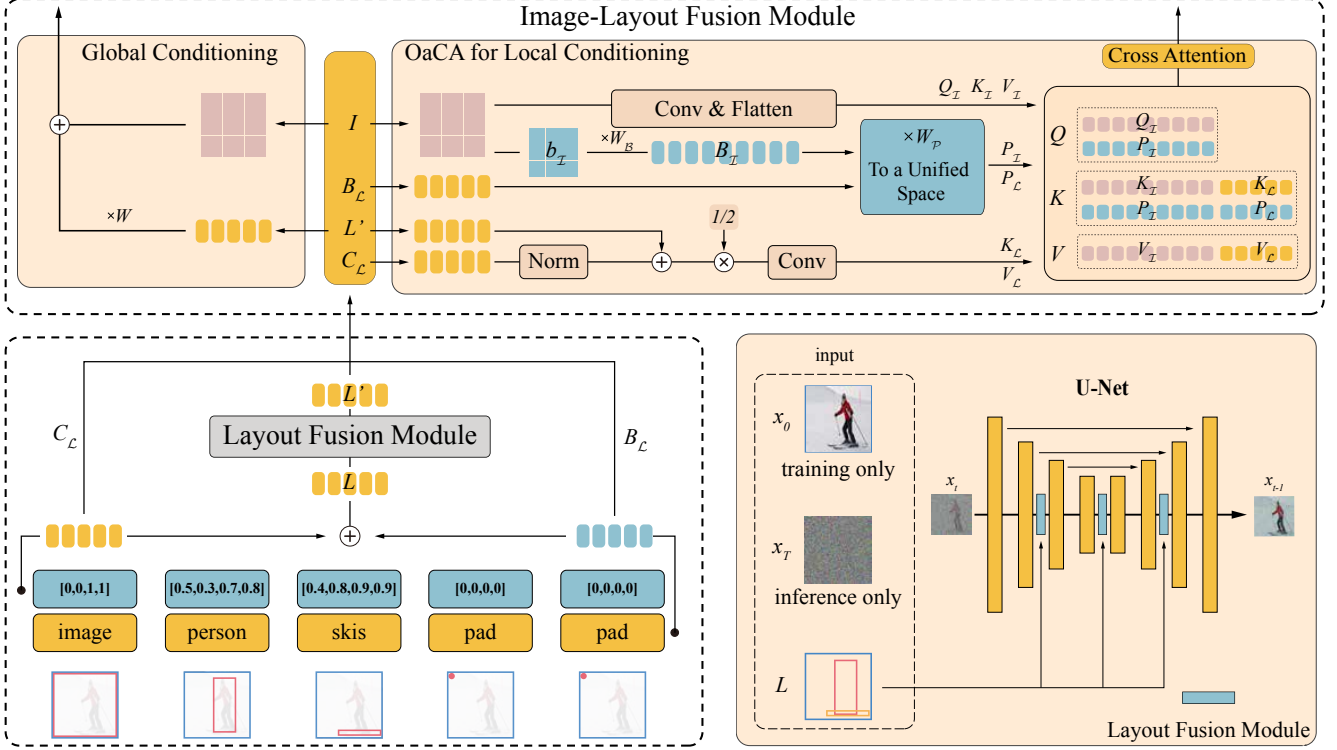


Figure 2. The whole pipeline of LayoutDiffusion. The layout that consisted of bounding box b and objects categories c is transformed into embedding $B_{\mathcal{L}}, C_{\mathcal{L}}, L$. Then Layout Fusion Module fuses layout embedding L to output the fused layout embedding L' . Finally, Image-Layout Fusion Module including direct addition used for global conditioning and Object-aware Cross Attention (OaCA) used for local conditioning, will fuse the layout related $B_{\mathcal{L}}, C_{\mathcal{L}}, L'$ and the image feature I at multiple resolutions.

$P_{\mathcal{L}}$ as follows:

$$B_{\mathcal{I}} = b_{\mathcal{I}} W_{\mathcal{B}} \quad (6)$$

$$P_{\mathcal{I}} = B_{\mathcal{I}} W_{\mathcal{P}} \quad (7)$$

$$P_{\mathcal{L}} = B_{\mathcal{L}} W_{\mathcal{P}} \quad (8)$$

, where $W_{\mathcal{B}} \in \mathbb{R}^{4 \times d_{\mathcal{L}}}$ is defined in Eq. 2 and works as a shared projection matrix that transforms the coordinates of bounding box into embedding of $d_{\mathcal{L}}$ dimension. $W_{\mathcal{P}} \in \mathbb{R}^{d_{\mathcal{L}} \times d_{\mathcal{I}}}$ is the projection matrix that transforms the B to the positional Embedding P .

Pointwise Addition for Global Conditioning. With the help of LFM in Eq. 4, O'_1 can be considered as a global information of the entire layout, and $O'_i (i \in [2, k])$ is considered as the local information embedding of single object along with the other related objects. One of the easiest ways to condition the layout in the image is to directly add O'_1 , the global information of the layout, to the multiple resolution of image features. Specifically, the condition process can be defined as

$$I' = I + O'_1 W \quad (9)$$

, where $W \in \mathbb{R}^{d_{\mathcal{L}} \times d_{\mathcal{I}}}$ is a projection matrix and I' is the image feature conditioned with global embedding of layout.

Object-aware Cross Attention for Local Conditioning.

Cross attention is successfully applied in [25] to condition text into image feature, where the sequence of the image patch is used as the query and the concatenated sequence of the image patch and text is applied as key and value. The equation of cross-attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (10)$$

, where Q, K, V represent the embeddings of query, key, and value, respectively. In the following paper, we will use the subscript image and layout to represent the image patch feature and layout feature, respectively.

In text-to-image generation, each token in the text sequence is a word. The aggregation of these words constitutes the semantics of a sentence. After the transformer encoder, the first token in text sequence is well-semantic information that generalizes the whole text but may not reverse the semantic meaning of each word. However, the loss in information of one token is relatively serious in layout rather than in text. Each token in the layout sequence is a single object with a specific category, size, and position. The loss of information on a layout token will directly lead

to a missing or wrong object in the generated image pixel space.

Therefore, we take into account the fusion of locations, size, and category of objects and define our object-aware cross-attention (OaCA) as

$$Q = \Psi_1(Q_{\mathcal{I}}, Q_{\mathcal{L}}) \quad (11)$$

$$K = \Psi_1(\Psi_2(K_{\mathcal{I}}, K_{\mathcal{L}}), \Psi_2(P_{\mathcal{I}}, P_{\mathcal{L}})) \quad (12)$$

$$V = \Psi_2(V_{\mathcal{I}}, V_{\mathcal{L}}) \quad (13)$$

, where the query $Q \in \mathbb{R}^{hw \times 2d_{\mathcal{I}}}$, $K \in \mathbb{R}^{(hw+k) \times 2d_{\mathcal{I}}}$, and $V \in \mathbb{R}^{(hw+k) \times d_{\mathcal{I}}}$. Ψ_1 and Ψ_2 denote concatenation on the dimension of the channel and length of the sequence, respectively.

We first construct the key and value of the layout:

$$K_{\mathcal{L}}, V_{\mathcal{L}} = \text{Conv}(\frac{1}{2}(\text{Norm}(C_{\mathcal{L}}) + L')) \quad (14)$$

, where $K_{\mathcal{L}}, V_{\mathcal{L}} \in \mathbb{R}^{k \times d_{\mathcal{I}}}$ and Conv is the convolution operation. The embedding of key and value in the layout is related to the category embedding $C_{\mathcal{L}}$ and the fused layout embedding L' . $C_{\mathcal{L}}$ focuses on the category information of layout and L' concentrates on the comprehensive information of both the object itself and other objects that may have a relationship with it. By averaging between L' and $C_{\mathcal{L}}$, we can obtain both the general information of the object and also emphasize the category information of the object.

We construct the query, key, and value of the image feature as follows:

$$Q_{\mathcal{I}}, K_{\mathcal{I}}, V_{\mathcal{I}} = \text{Conv}(\text{Norm}(I)) \quad (15)$$

3.4. Layout-conditional Diffusion Model

Here, we follow the Gaussian diffusion models improved by [10, 38]. Given a data point sampled from a real data distribution $x_0 \sim q(x_0)$, a forward diffusion process is defined by adding small amount of Gaussian noise to the x_0 in T steps:

$$q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)\mathbf{I}) \quad (16)$$

If the total noise added throughout the Markov chain is large enough, the x_T will be well approximated by $\mathcal{N}(0, \mathbf{I})$. If we add noise at each step with a sufficiently small magnitude $1 - \alpha_t$, the posterior $q(x_{t-1}|x_t)$ will be well approximated by a diagonal Gaussian. This nice property ensures that we can reverse the above forward process and sample from $x_T \sim \mathcal{N}(0, \mathbf{I})$, which is a Gaussian noise. However, since the entire dataset is needed, we are unable to easily estimate the posterior. Instead, we have to learn a model $p_{\theta}(x_{t-1}|x_t)$ to approximate it:

$$p_{\theta}(x_{t-1}|x_t) := \mathcal{N}(\mu_{\theta}(x_t), \Sigma_{\theta}(x_t)) \quad (17)$$

Instead of using the tractable variational lower bound (VLB) in $\log p_{\theta}(x_0)$, Ho et al. [10] proposed to reweight the terms of the VLB to optimize a surrogate objective. Specifically, we first add t steps of Gaussian noise to a clean sample x_0 to generate a noised sample $x_t \sim q(x_t|x_0)$. Then train a model ϵ_{θ} to predict the added noise using the following loss:

$$\mathcal{L} := E_{t \sim [1, T], x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0, \mathbf{I})} [\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2] \quad (18)$$

, which is a standard mean-squared error loss.

To support the layout condition, we apply classifier-free guidance, a technique proposed by Ho et al. [11] for conditional generation that requires no additional training of the classifier. It is accomplished by interpolating between predictions of a diffusion model with and without condition input. For the condition of layout, we first construct a padding layout $l_{\phi} = \{o_l, o_p, \dots, o_p\}$. During training, the condition of layout l of diffusion model will be replaced with l_{ϕ} with a fixed probability. When sampling, the following equation is used to sample a layout-conditional image:

$$\hat{\epsilon}_{\theta}(x_t, t|l) = (1 - s) \cdot \epsilon_{\theta}(x_t, t|l_{\phi}) + s \cdot \epsilon_{\theta}(x_t, t|l) \quad (19)$$

, where the scale s can be used to increase the gap between $\epsilon_{\theta}(x_t, t|l_{\phi})$ and $\epsilon_{\theta}(x_t, t|l)$ to enhance the strength of conditional guidance.

To further improve the user experience of LayoutDiffusion, we also make several optimizations on the speed of the classifier-free sampling process and could significantly outperform the SOTA models in 25 iterations. Specifically, we adapt DPM-solver [22] for the conditional classifier-free sampling, a fast dedicated high-order solver for diffusion ODEs [39] with the convergence order guarantee, to accelerate the conditional sampling speed.

4. Experiments

In this section, we evaluate our LayoutDiffusion on different benchmarks in terms of various metrics. First, we introduce the datasets and evaluation metrics. Second, we show the qualitative and quantitative results compared with other strategies. Finally, some ablation studies and analysis are also mentioned. More details can be found in Appendix, including model architecture, training hyperparameters, reproduction results, more experimental results and visualizations.

4.1. Datasets

We conduct our experiments on two popular datasets, COCO-Stuff [5] and Visual Genome [19].

COCO-Stuff has 164K images from COCO 2017, of which the images contain bounding boxes and pixel-level segmentation masks for 80 categories of thing and 91 categories

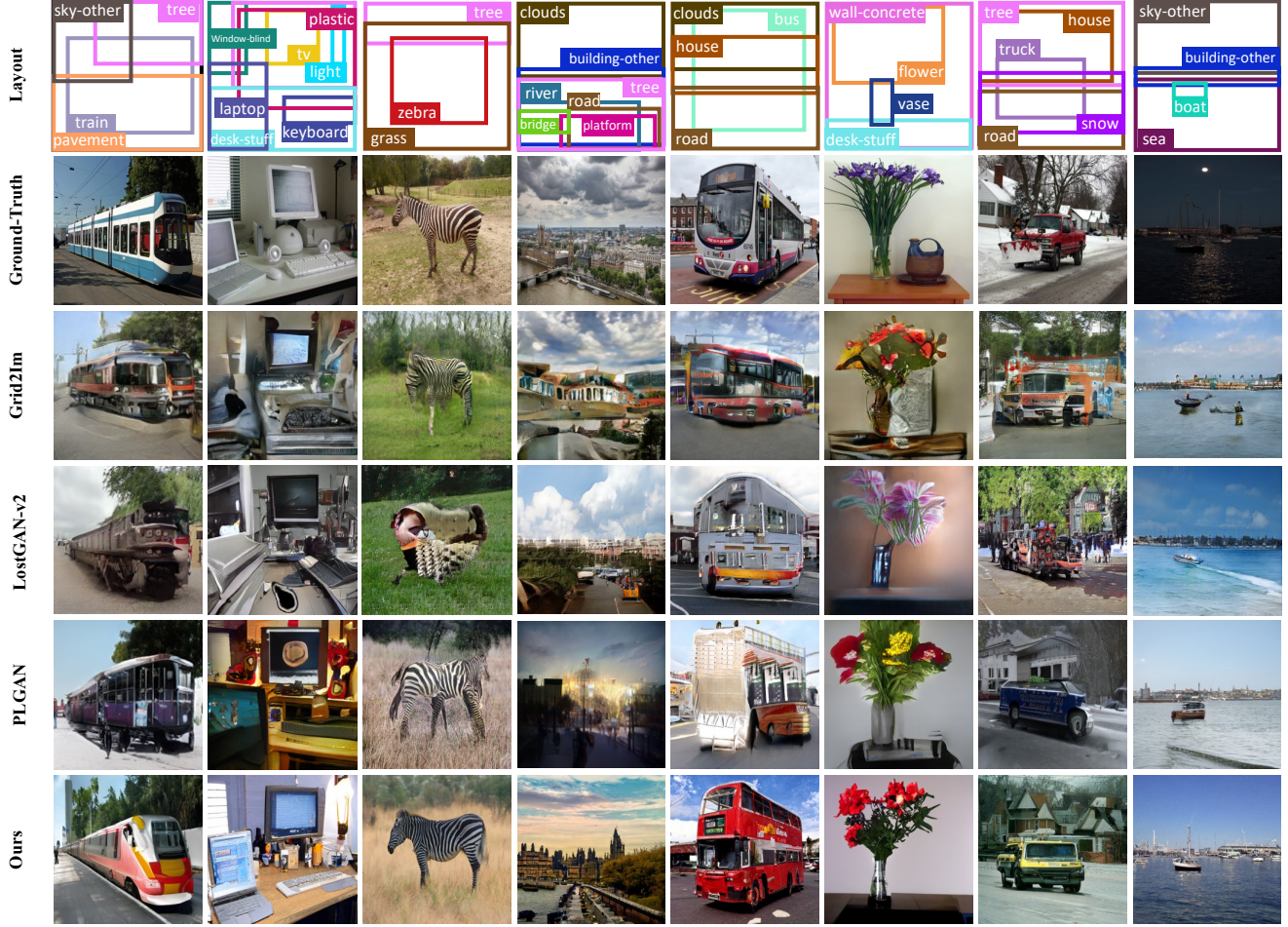


Figure 3. Visualization of comparison with SOTA methods on COCO-stuff 256×256 . LayoutDiffusion has better generation quality and stronger controllability compared to the other methods.

Methods	COCO-stuff					VG		
	FID ↓	IS ↑	DS ↑	CAS ↑	YOLOScore ↑	FID ↓	DS ↑	CAS ↑
128×128								
Grid2Im [2]	59.50	12.50±0.30	0.28±0.11	4.05	6.80	-	-	-
LostGAN-v2 [41]	24.76	14.21±0.40	0.45±0.09	39.91	13.60	29.00	0.42±0.09	29.74
PLGAN [45]	22.70	15.60±0.30	0.16±0.72	38.70	13.40	20.62	-	-
LayoutDiffusion	16.57	20.17±0.56	0.47±0.09	43.60	27.00	16.35	0.49±0.09	36.45
256×256								
Grid2Im [2]	65.20	16.40±0.70	0.34±0.13	4.81	9.70	-	-	-
LostGAN-v2 [41]	31.18	18.01±0.50	0.56±0.10	40.00	17.50	32.08	0.53±0.10	34.48
PLGAN [45]	29.10	18.90±0.30	0.52±0.10	37.65	14.40	28.06	-	-
LayoutDiffusion	15.61	28.36±0.75	0.57±0.10	47.74	32.00	15.63	0.59±0.10	48.90

Table 1. Quantitative results on COCO-stuff [5] and VG [19]. The proposed diffusion method has made great progress in all evaluation metrics, showing better quality, controllability, diversity, and accuracy than previous works. For COCO-stuff, we evaluate on 3097 layout and sample 5 images for each layout. For VG, we evaluate on 5096 layout and sample 1 image for each layout. We also report reproduction scores of previous works in Appendix.

LFM	OaCA	CA	FID ↓	IS ↑	DS ↑	CAS ↑	YOLOScore ↑
			29.94 (+13.37)	13.59±0.29 (-6.58)	0.70±0.08 (+0.23)	3.83 (-39.77)	0.00 (-27.00)
✓			17.06 (+0.49)	19.21±0.53 (-0.96)	0.52±0.09 (+0.05)	30.86 (-12.74)	6.90 (-20.10)
	✓		16.76 (+0.19)	19.57±0.40 (-0.60)	0.48±0.09 (+0.01)	40.67 (-2.93)	18.80 (-8.20)
✓		✓	16.46 (-0.11)	19.79±0.40 (-0.38)	0.48±0.10 (+0.01)	42.47 (-1.13)	23.60 (-3.40)
✓	✓		16.57	20.17±0.56	0.47±0.09	43.60	27.00

Table 2. Ablation study of **Layout Fusion Module (LFM)**, **Object-aware Cross Attention (OaCA)**, **Cross Attention (CA)**. We use the model trained for 300,000 iterations on COCO-stuff 128×128. The value in brackets denotes the discrepancy to our proposed method(+LFM+OaCA), where red denotes better and green denotes worse.

of stuff, respectively. Following the settings of LostGAN-v2 [41], we use the COCO 2017 Stuff Segmentation Challenge subset that contains 40K / 5k / 5k images for train / val / test-dev set, respectively. We use images in the train and val set with 3 to 8 objects that cover more than 2% of the image and not belong to *crowd*. Finally, there are 25,210 train and 3,097 val images.

Visual Genome collects 108,077 images with dense annotations of objects, attributes, and relationships. Following the setting of SG2Im [14], we divide the data into 80%, 10%, 10% for the train, val, test set, respectively. We select the object and relationship categories occurring at least 2000 and 500 times in the train set, respectively, and select the images with 3 to 30 bounding boxes and ignoring all small objects. Finally, the training / validation / test set will have 62565 / 5062 / 5096 images, respectively.

4.2. Evaluation Metrics & Protocols

We use five metrics to evaluate the quality, diversity, and controllability of generation.

Fréchet Inception Distance (FID) [9] shows the overall visual quality of the generated image by measuring the difference in the distribution of features between the real images and the generated images on an ImageNet-pretrained Inception-V3 [42] network.

Inception Score (IS) [35] uses an Inception-V3 [42] pre-trained on ImageNet network to compute the statistical score of the output of the generated images.

Diversity Score (DS) calculates the diversity between two generated images of the same layout by comparing the LPIPS [49] metric in a DNN feature space between them.

Classification Score (CAS) [29] first crops the ground truth box area of images and resizing them at a resolution of 32×32 with their class. A ResNet-101 [8] classifier is trained with generated images and tested on real images.

YOLOScore [20] evaluates 80 thing categories bbox mAP on generated images using a pretrained YOLOv4 [4] model, and shows the precision of control in one generated model.

In summary, FID and IS show the generation quality, DS shows the diversity, CAS and YOLOScore represent the controllability. We follow the architecture of ADM [7], which is mainly a UNet. All experiments are conducted on 32 NVIDIA 3090s with mixed precision training [24]. We

set batch size 24, learning rate 1e-5. We adopt the fixed linear variance schedule. More details can be found in the Appendix.

4.3. Qualitative results

Comparison of generated 256 × 256 images on the COCO-Stuff [5] with our method and previous works [2, 41, 45] is shown in Fig. 3.

method	FID ↓	N_{parms}	Throughput images / s	First stage V100 days	Cond stage V100 days
LDM-8 (100 steps)	42.06	345M	0.457	66	3.69
LDM-4 (200 steps)	40.91	306M	0.267	29	95.49
ours-small (25 steps)	36.16	142M	0.608	-	75.83
ours (25 steps)	31.68	569M	0.308	-	216.55

Table 3. Comparison with SOTA diffusion-based methods LDM on COCO-stuff 256×256. We generate the same 2048 images of LDM for a fair comparison.

LayoutDiffusion generates more accurate high quality images, which has more recognizable and accurate objects corresponding to their layouts. Grid2Im [2], LostGAN-v2 [41] and PLGAN [45] generate images with distorted and unreal objects.

Especially when input a set of multiple objects with complex relationships, previous work can hardly generate recognizable objects in the position corresponding to layouts. For example, in Fig. 3 (a), (c), and (e), the main objects (e.g. train, zebra, bus) in images are poorly generated in previous work, while our LayoutDiffusion generates well. In Fig. 3 (b), only our LayoutDiffusion generates the laptop in the right place. The images generated by our LayoutDiffusion are more sensorially similar to the real ones.

We show the diversity of LayoutDiffusion in Fig. 4. Images from the same layouts have high quality and diversity (different lighting, textures, colors, and details).

We continuously add an additional layout from the initial layout, the one in the upper left corner, as shown in Fig. 5. In each step, LayoutDiffusion adds the new object in very precise locations with consistent image quality, showing user-friendly interactivity.

4.4. Quantitative results

Tab. 1 provides the comparison among previous works and our method in FID, IS, DS, CAS and YOLOScore. Compared to the SOTA method, the proposed method achieves the best performance in comparison.

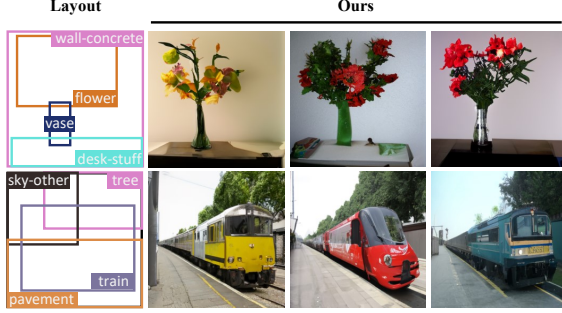


Figure 4. The diversity of LayoutDiffusion. Each row of images are from the same layout and have great difference.

In overall generation quality, our LayoutDiffusion outperforms the SOTA model by 46.35% and 29.29% at most in FID and IS, respectively. While maintaining high overall image quality, we also show precise and accurate controllability, LayoutDiffusion outperforms the SOTA model by 122.22% and 41.82% at most on YOLOSore and CAS, respectively. As for diversity, our LayoutDiffusion still achieves 11.30% improvement at most according to the DS. Experiments on these metrics show that our methods can successfully generate the higher-quality images with better location and quantity control.

In particular, we conduct experiments compared to LDM [32] in Tab. 3. “Ours-small” uses comparable GPU resources to have better FID performance with much fewer parameters and better throughput compared to LDM-8 when “Ours-small” outperforms LDM-4 in all respects. The results of “Ours” indicate that LayoutDiffusion can have better FID performance, 31.6, at a higher cost. From these results, LayoutDiffusion always achieves better performance at different cost levels compared with LDM [32].

4.5. Ablation studies

We validate the effectiveness of LFM and OaCA in Tab. 2, using the evaluation metrics in Sec. 4.2. The significant improvement on FID, IS, CAS, and YOLOSore proves that the application of LFM and OaCA allows for higher generation quality and diversity, along with more controllability. Furthermore, when applying both, considerable performance, 13.37 / 6.58 / 39.77 / 27.00 on FID / IS / CAS / YOLOSore, is gained.

An interesting phenomenon is that the change of the Diversity Score (DS) is in the opposite direction of other metrics. This is because DS, which stands for diversity, is physically the opposite of the controllability represented by other metrics such as CAS and YOLOSore. The precise control offered on generated image leads to more constraints on diversity. As a result, the Diversity Score (DS) has a slight drop compared to the baseline.

5. Limitations & Societal Impacts

Limitations. Despite the significant improvements in various metrics, it is still difficult to generate a realistic im-



Figure 5. The interactivity of LayoutDiffusion. We add extra layout continuously, and the new objects are also with high quality.

age with no distortion and overlap, especially for a complex multi-object layout. Moreover, the model is trained from scratch in the specific dataset that requires detection labels. How to combine text-guided diffusion models and inherit parameters pre-trained on massive text-image datasets remains a future research.

Societal Impacts. Trained on the real-world datasets such as COCO [5] and VG [19], LayoutDiffusion has the powerful ability to learn the distribution of data and we should pay attention to some potential copyright infringement issues.

6. Conclusion

In this paper, we have proposed a one-stage end-to-end diffusion model named LayoutDiffusion, which is novel for the task of layout-to-image generation. With the guidance of layout, the diffusion model allows more control over the individual objects while maintaining higher quality than the prevailing GAN-based methods. By constructing a structural image patch with region information, we regard each patch as a special object and accomplish the difficult multimodal image-layout fusion in a unified form. Specifically, Layout Fusion Module and Object-aware Cross Attention are proposed to model the relationship among multiple objects and fuse the patched image feature with layout at multiple resolutions, respectively. Experiments in challenging COCO-stuff and Visual Genome (VG) show that our proposed method significantly outperforms both state-of-the-art GAN-based and diffusion-based methods in various evaluation metrics.

Acknowledgements. This work is supported in part by National Natural Science Foundation of China under Grant U20A20222, National Science Foundation for Distinguished Young Scholars under Grant 62225605, National Key Research and Development Program of China under Grant 2020AAA0107400, Research Fund of ARC Lab, Tencent PCG, Zhejiang – Singapore Innovation and AI Joint Research Lab, Ant Group through CCF-Ant Research Fund, and sponsored by CCF-AFSG Research Fund, CAAI-HUAWEI MindSpore Open Fund as well as CCF-Zhipu AI Large Model Fund(CCF-Zhipu202302).

Appendix for LayoutDiffusion

A. More Visualizations

A.1. Controllable Layout Edit by modifying objects in number, position, size, and category

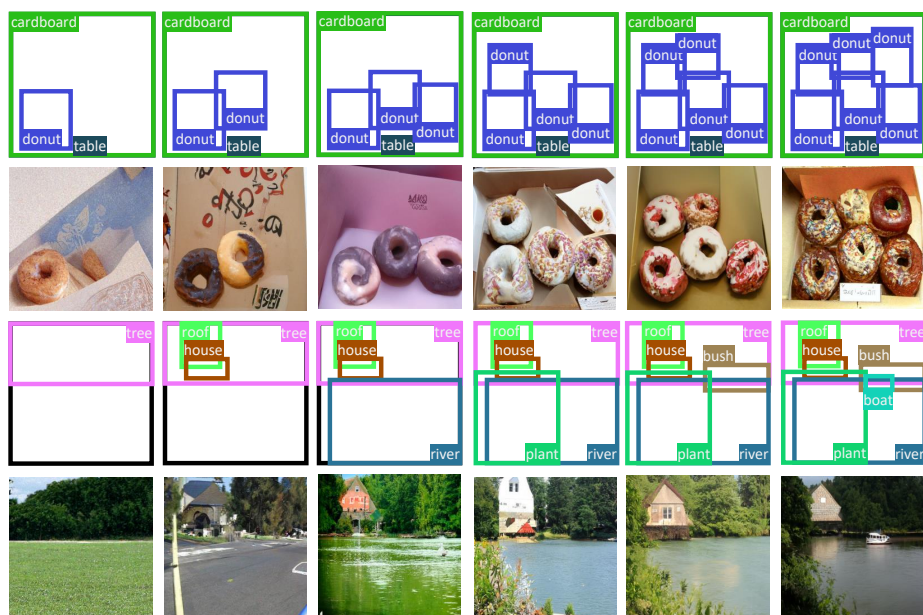


Figure 6. Layout edit by adding objects.

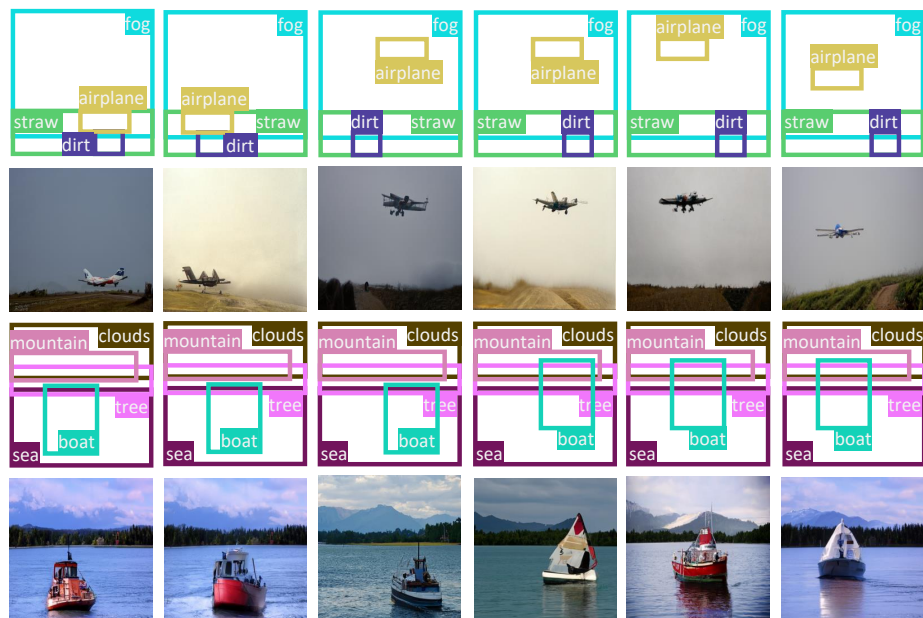


Figure 7. Layout edit by modifying the position of objects.



Figure 8. Layout edit by modifying the size of objects.

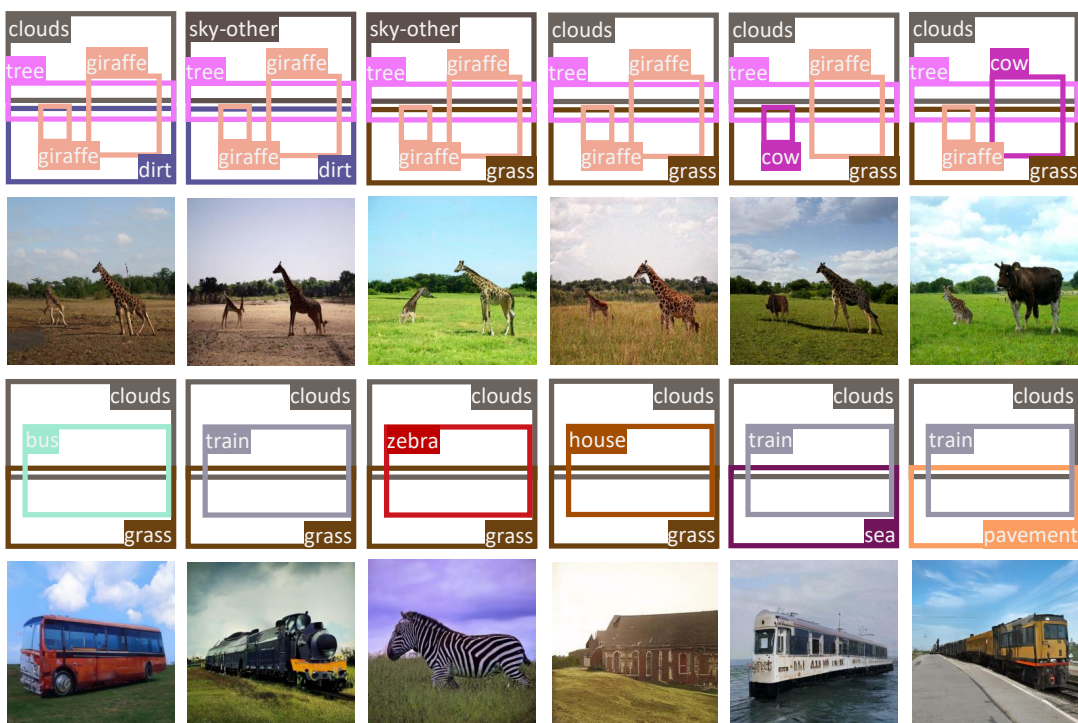


Figure 9. Layout edit by modifying the categories of objects.

A.2. More visualizations on COCO-stuff

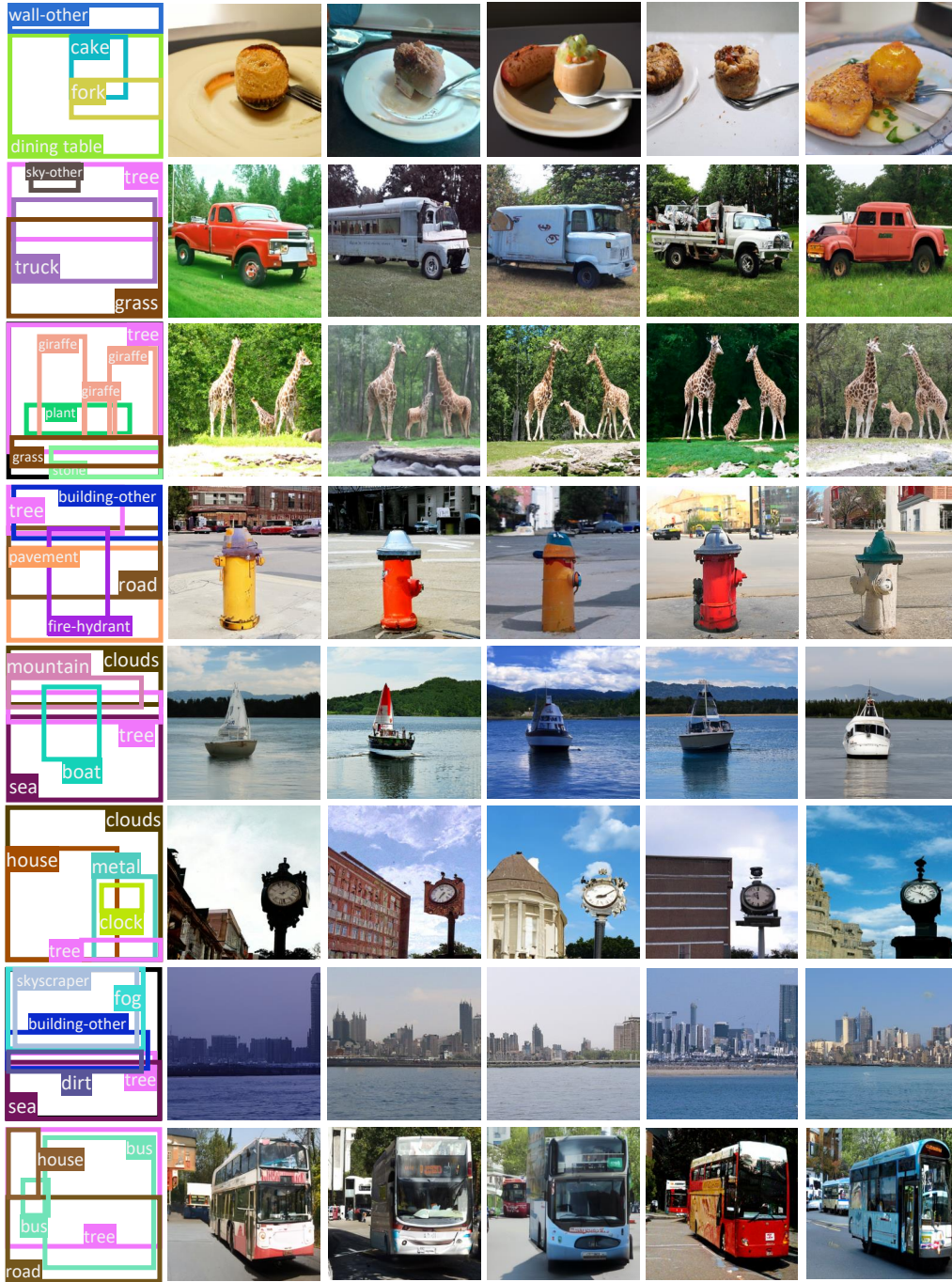


Figure 10. More visualizations on COCO-stuff 256×256 . LayoutDiffusion is trained by 1.15M iterations, and sample images using scale=1.0 and dpm-solver 25 steps. The COCO image IDs (from top to bottom) are 85195, 174004, 296969, 338560, 451090, 512248, 573008, 574425.

A.3. More visualizations on Visual Genome

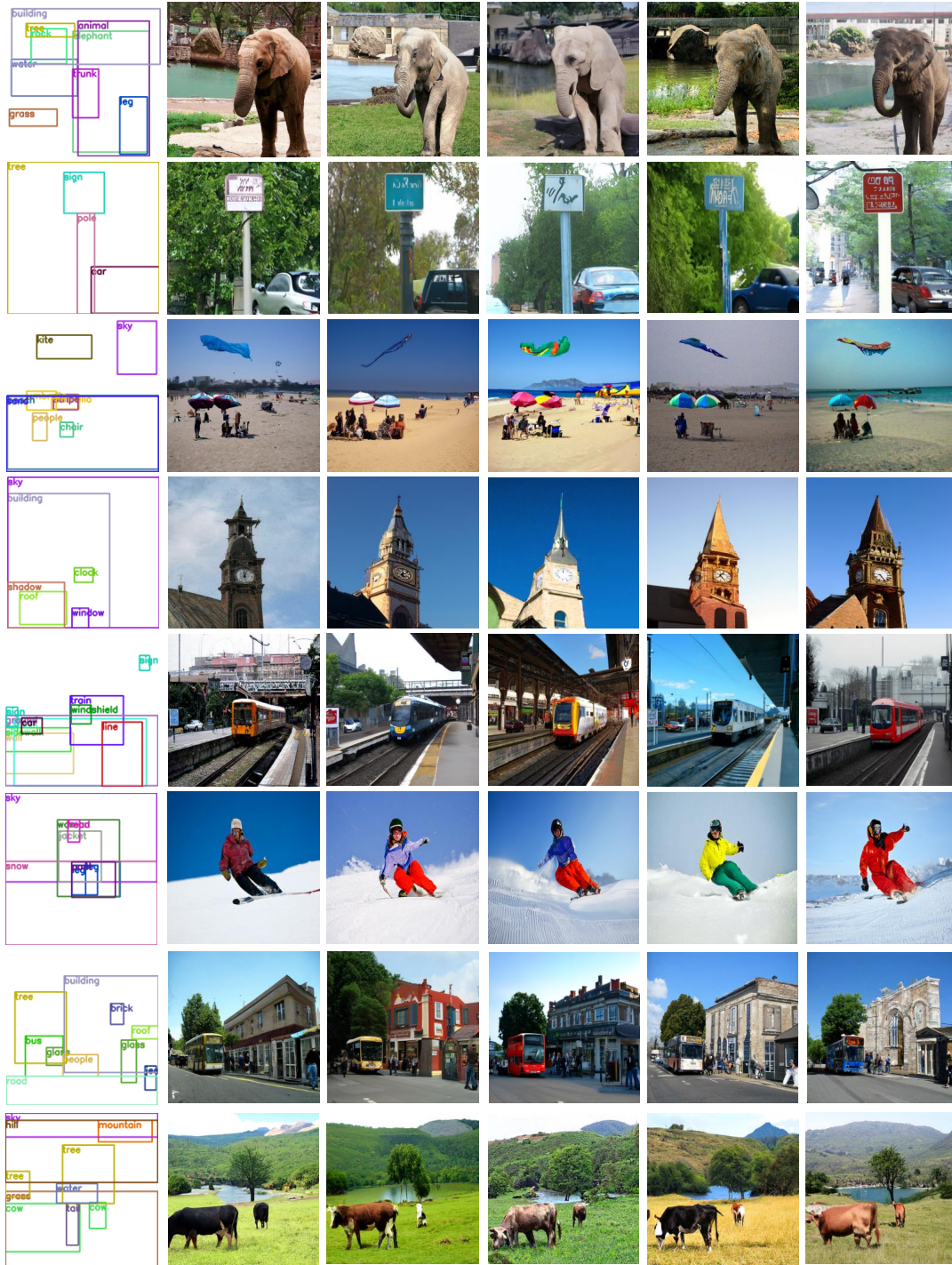


Figure 11. More visualizations on Visual Genome 256×256 . LayoutDiffusion is trained by 1.45M iterations, and sample images using scale=1.0 and dpm-solver 25 steps. The VG image IDs (from top to bottom) are 2380568, 2382403, 2382599, 2383021, 2383488, 2385225, 2385290, 2385812.

A.4. More comparison with previous methods

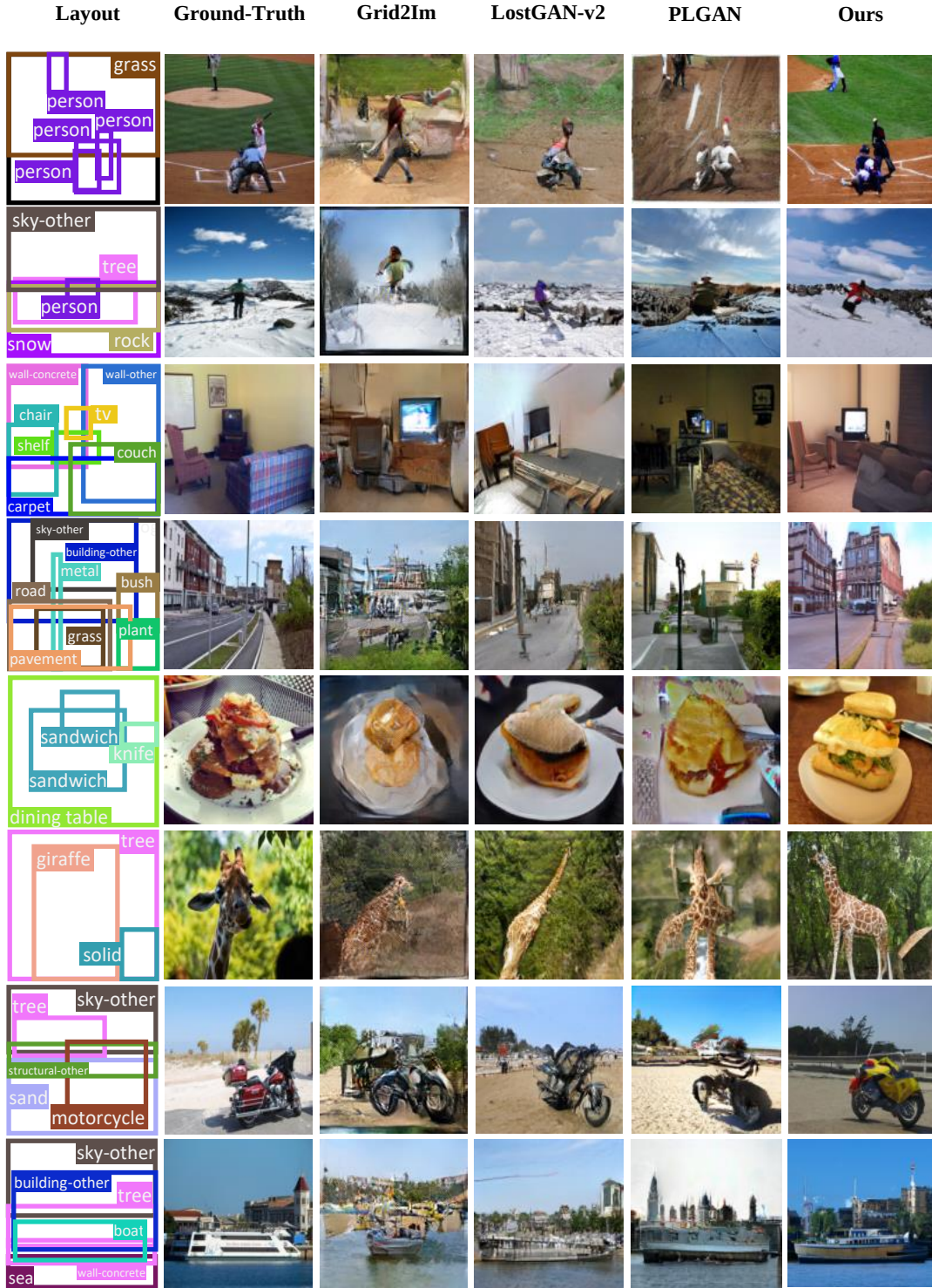


Figure 12. More comparison with previous methods on COCO-stuff 128×128 . LayoutDiffusion is trained by 300K iterations, and sample images using scale=0.6 and dpm-solver 25 steps. The COCO image IDs (from top to bottom) are 2153, 2352, 4495, 6723, 10583, 17031, 18737, 543300.

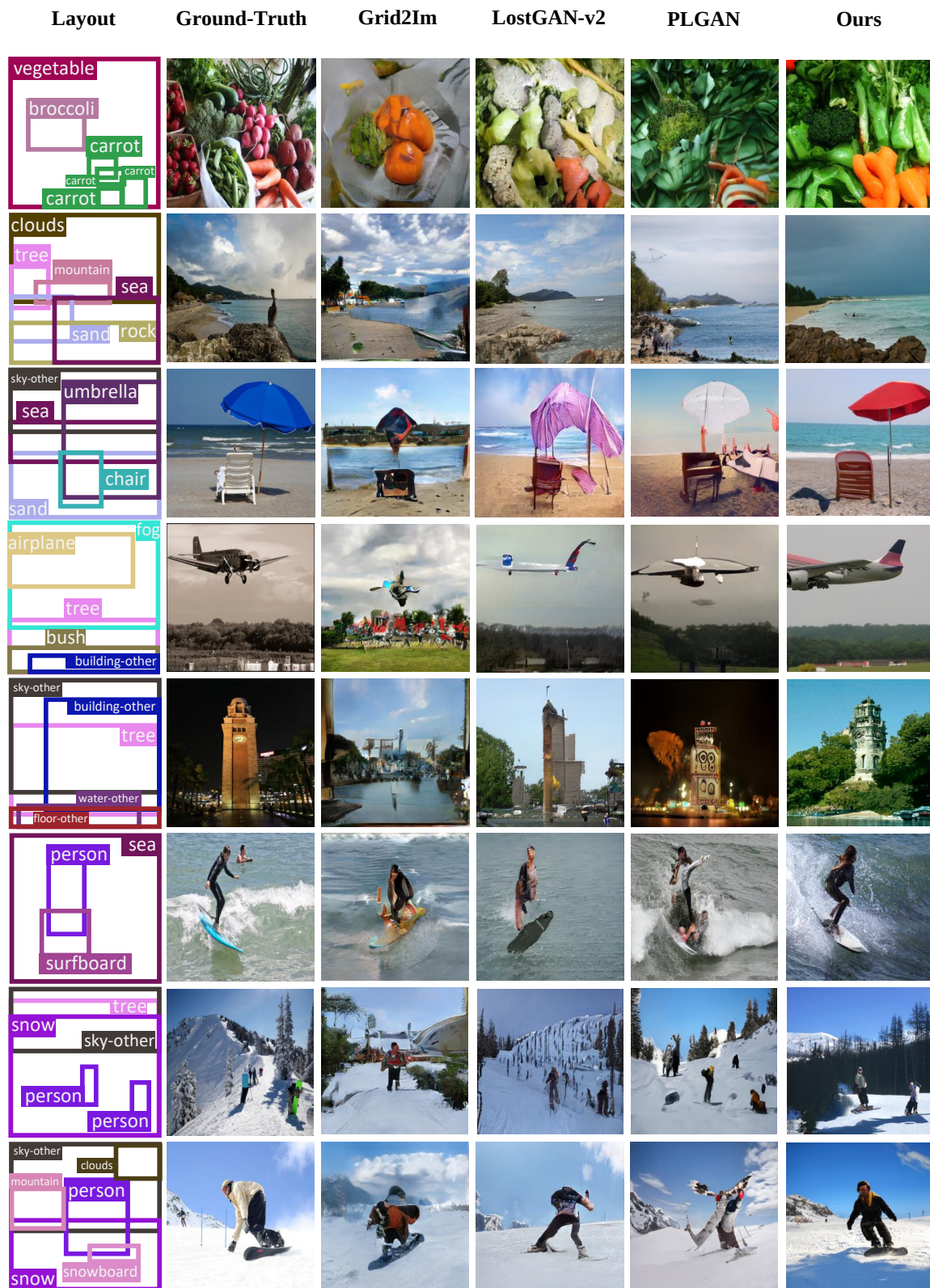


Figure 13. More comparison with previous methods on COCO-stuff 256×256 . LayoutDiffusion is trained by 1.15M iterations, and sample images using scale=1.0 and dpm-solver 25 steps. The COCO image IDs (from top to bottom) are 23781, 55299, 84477, 137950, 243034, 252701, 341719, 350405.

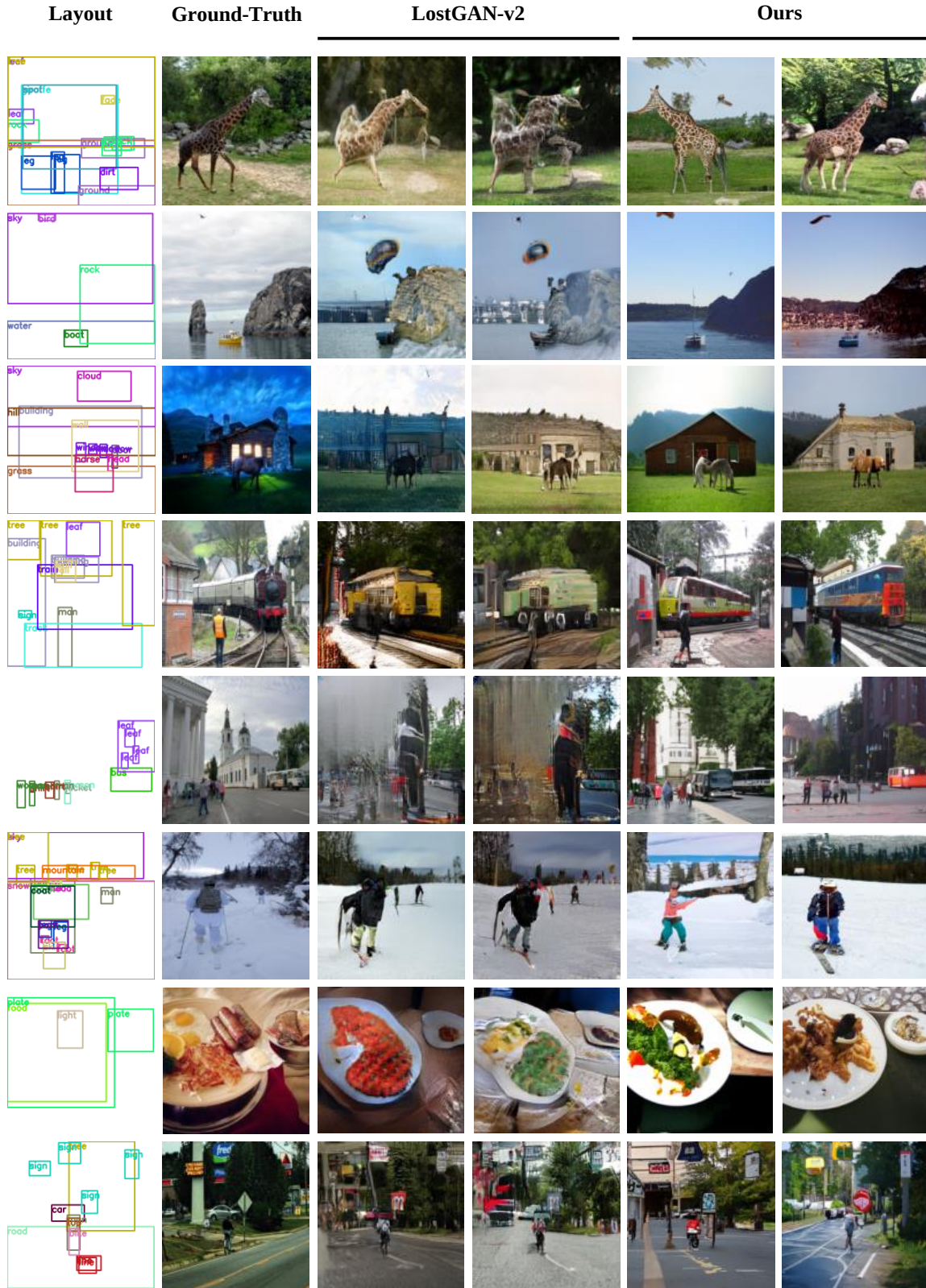


Figure 14. More comparison with previous methods on VG 128×128 . LayoutDiffusion is trained by 300K iterations, and sample images using scale=0.5 and dpm-solver 25 steps. The VG image IDs (from top to bottom) are 107945, 150280, 150409, 1160185, 1591817, 1592132, 2341006, 2341475.

A.5. Different Scales

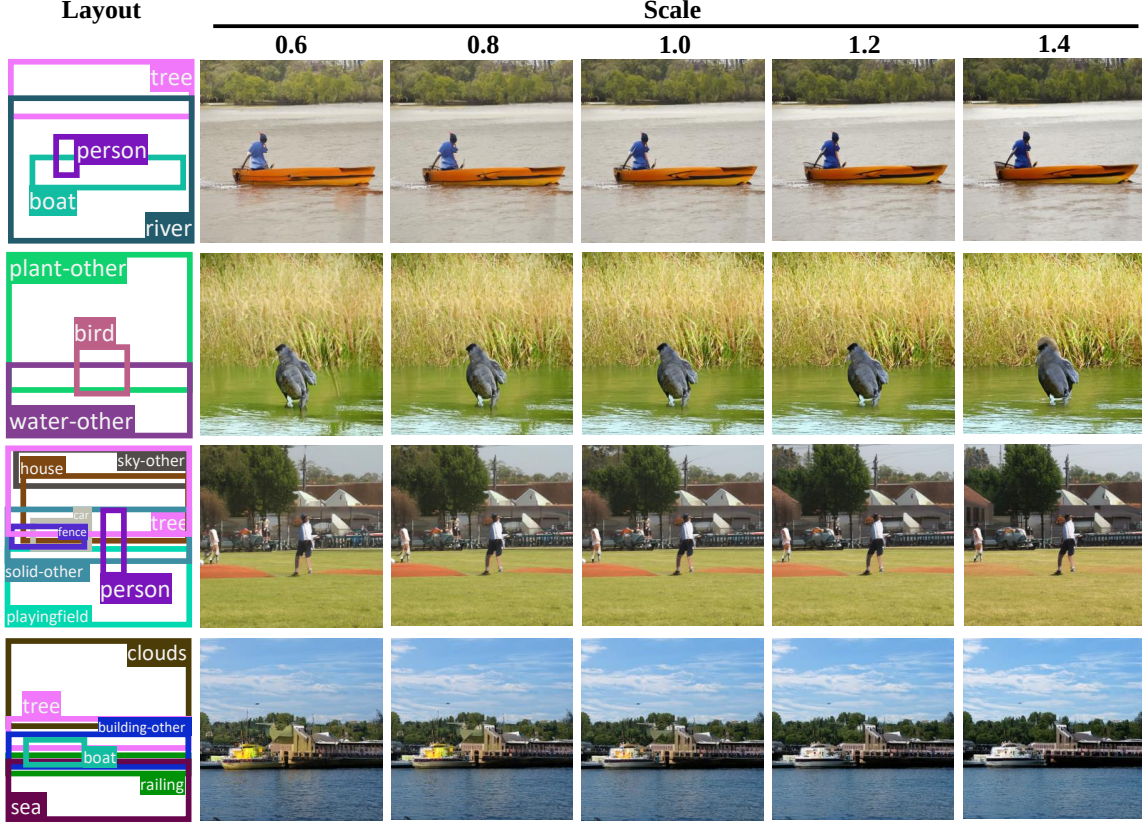


Figure 16. Visualizations on COCO-stuff 256×256 sampled with different scale. LayoutDiffusion is trained by 1.15M iterations, and sample images using dpm-solver 25 steps. The COCO image IDs (from top to bottom) are 472298, 475223, 504415, 513181.

B. Details of Diffusion Models

B.1. Denoising Diffusion Probabilistic Model (from ADM-G)

In this section, we will review the formulation of Gaussian diffusion models introduced by DDPM [10]. A data point is defined as $x_0 \sim q(x_0)$. By gradually adding noise to the clean data x_0 , we can obtain the noised samples from x_1 to x_T , where T denotes the maximum steps. Specifically, Gaussian noise according to some variance schedule given by β_t is added to x_{t-1} in step t of the Markovian noising process q :

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I}) \quad (20)$$

Due to the convenient nature of Gaussian noise, we do not need to apply t times of $q(x_t|x_{t-1})$ repeatedly to sample from $x_t \sim q(x_t|x_0)$. Instead, $q(x_t|x_0)$ can be directly sampled from a Gaussian distribution:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (21)$$

$$= \sqrt{\bar{\alpha}_t}x_0 + \epsilon\sqrt{1 - \bar{\alpha}_t}, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (22)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=0}^t \alpha_s$. The noise variance for an arbitrary timestep is defined as $1 - \bar{\alpha}_t$ in Eq. (22), and we could equivalently use this to define the noise schedule instead of β_t . DDPM [10] notes that the posterior $q(x_{t-1}|x_t, x_0)$ is

also a Gaussian using with mean $\tilde{\mu}_t(x_t, x_0)$ and variance $\tilde{\beta}_t$, and is defined as follows:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I}) \quad (23)$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t \quad (24)$$

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t \quad (25)$$

If the total noise added throughout the markov chain is large enough when $T \rightarrow \infty$ and correspondingly $\beta_t \rightarrow 0$, the x_T will be well approximated by $\mathcal{N}(0, \mathbf{I})$. This nice property ensures that we can reverse the above forward process and sample from $x_T \sim \mathcal{N}(0, \mathbf{I})$, which is a Gaussian noise. However, since the entire dataset is needed, we cannot easily estimate the posterior $q(x_{t-1}|x_t, x_0)$. Instead, we have to learn a model $p_\theta(x_{t-1}|x_t)$ to approximate it:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (26)$$

To ensure that the estimated $p_\theta(x_{t-1}|x_t)$ can learn the true data distribution $q(x_0)$, we can optimize the following variational lower bound L_{vib} for $p_\theta(x_0)$:

$$L_{\text{vib}} = L_0 + L_1 + \dots + L_{T-1} + L_T \quad (27)$$

$$L_0 = -\log p_\theta(x_0|x_1) \quad (28)$$

$$L_{t-1} = D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t)) \quad (29)$$

$$L_T = D_{KL}(q(x_T|x_0) || p(x_T)) \quad (30)$$

Although the above objective is well justified, DDPM [10] applied a different objective that produces better samples in practice. Specifically, they do not directly predict $\mu_\theta(x_t, t)$ as the output of a neural network, but instead train a model $\epsilon_\theta(x_t, t)$ to predict ϵ from Equation 22. This simplified objective is defined as follows:

$$L_{\text{simple}} = E_{t \sim [1, T], x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0, \mathbf{I})} [||\epsilon - \epsilon_\theta(x_t, t)||^2] \quad (31)$$

Then, we can derive $\mu_\theta(x_t, t)$ from $\epsilon_\theta(x_t, t)$ using the following substitution:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \quad (32)$$

Note that $\Sigma_\theta(x_t, t)$ is not learned in L_{simple} in DDPM [10] and is fixed as a constant such as $\beta_t \mathbf{I}$ or $\tilde{\beta}_t \mathbf{I}$, corresponding to upper and lower bounds for the true reverse step variance [37], respectively.

B.2. Classifier-free Method for Layout-conditional Training and Sampling

Instead of training a separate classifier model, Ho & Salimans [11] choose to train a diffusion model that allows for both conditional and unconditional sampling, where the unconditional diffusion model $p_\theta(x_t, t)$ is parameterized through a score estimator $\epsilon_\theta(x_t, t)$ and the conditional diffusion model $p_\theta(x_t, t|c)$ is parameterized through $\epsilon_\theta(x_t, t, c)$.

They use a single model to parameterize both conditional and unconditional models, where for the unconditional model they simply input a null token $\emptyset$ for the class identifier c when predicting the score, i.e. $\epsilon_\theta(x_t, t) = \epsilon_\theta(x_t, t, c = \emptyset)$. They jointly train this unified model by randomly replacing c with the unconditional class identifier $\emptyset$ with probability p_{uncond} . Then the conditional score estimate $\epsilon_\theta(x_t, t, c)$ is replaced with $\tilde{\epsilon}_\theta(x_t, t, c)$ using the following equation:

$$\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t, c) + s(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t)), \quad (33)$$

which can be considered as a linear combination of conditional and unconditional score estimates. s is the scale and Since Eq. (33) has no classifier gradient, no more gradient calculation in classifier guidance is needed during sampling. Furthermore, s can be changed to modify the effect of the condition.

In the layout-to-image generation, c is the layout l defined in Sec. 3.1. Layout Embedding and $\emptyset$ is the empty layout l_{pad} defined in Sec. 3.4. Layout-conditional Diffusion Model.

C. Implementation details

C.1. Hyperparameters

Dataset	COCO-stuff 256×256		COCO-stuff 128×128	VG 256×256	VG 128×128
Model	LayoutDiffusion	LayoutDiffusion-small	LayoutDiffusion	LayoutDiffusion	LayoutDiffusion
Layout-conditional Diffusion Model					
In Channels	3	3	3	3	3
Out Channels	6	6	6	6	6
Hidden Channels	256	128	256	256	256
Channel Multiply	1,1,2,2,4,4	1,1,2,2,4,4	1,1,2,3,4	1,1,2,2,4,4	1,1,2,3,4
Number of Residual Blocks	2	2	2	2	2
Dropout	0	0	0.1	0.1	0.1
Diffusion Steps	1000	1000	1000	1000	1000
Noise Schedule	linear	linear	linear	linear	linear
Layout-Image Fusion Module					
Downsampling Scale For Fusion	8,16,32	8,16,32	4,8,16	8,16,32	4,8,16
Resolution for Fusion	32,16,8	32,16,8	32,16,8	32,16,8	32,16,8
Fusion Method	OaCA	OaCA	OaCA	OaCA	OaCA
Number of Attention Blocks	1	1	1	1	1
Number of Heads	4	4	4	4	4
Layout Fusion Module					
Hidden Channels	256	128	256	256	256
Transformer Depth	6	4	6	6	6
Attention Method	Self-Attention	Self-Attention	Self-Attention	Self-Attention	Self-Attention
Number of Heads	8	8	8	8	8
Layout Embedding					
Embedding Dimension	256	128	256	256	256
Maximum Number of Objects	8	8	8	10	10
Maximum Number of Length	10	10	10	12	12
Maximum Number of Class Id	185	185	185	180	180
Training Hyperparameters					
Total Batch Size	32	32	64	32	64
Number of GPUs	8	8	8	8	8
Learning Rate	1e-5	1e-5	2e-5	1e-5	2e-5
Mixed Precision Training	Yes	Yes	Yes	Yes	Yes
Weight Decay	0	0	0	0	0
EMA Rate	0.9999	0.9999	0.9999	0.9999	0.9999
Classifier-free Dropout	0.2	0.2	0.2	0.2	0.2
Iterations	1.15M	1.4M	300K	1.45M	300K

Table 4. Hyperparameters for the proposed LayoutDiffusion in Sec. 4.4. Quantitative results. All trained on eight RTX 3090.

Model	LayoutDiffusion	– LFM	– OaCA	– OaCA, + CA	– LFM, – OaCA
Fusion Method	OaCA	OaCA	-	CA	-
Transformer Depth	6	0	6	6	0
Iterations	300K	300K	300K	300K	300K
FID	16.57	+ 0.19	+ 0.49	- 0.11	+ 13.37
DS	0.47	+ 0.01	+ 0.05	+ 0.01	+ 0.23
CAS	43.60	- 2.93	- 12.74	- 1.13	- 39.77
YOLOScore	27.00	- 8.20	- 20.10	- 3.4	- 27.0

Table 5. Hyperparameters for the ablation study in Sec. 4.5 .

C.2. Analysis of Training Resources and Sampling Speed

method	FID ↓	N_{parms}	Throughout images / s	Compression stage V100 days	Diffusion stage V100 days	Total V100 days
LDM-8 (100 steps)	42.06	345M	0.457	66	3.69	69.69
LDM-4 (200 steps)	40.91	306M	0.267	29	95.49	124.49
LayoutDiffusion-small (25 steps)	36.16	142M	0.608	-	75.83	75.83
LayoutDiffusion (25 steps)	31.68	569M	0.308	-	216.55	216.55

Table 6. Comparison with SOTA diffusion-based methods LDM on COCO-stuff 256×256 . We generate the same 2048 images of LDM for a fair comparison. LDM is sampled using DDIM [37] and LayoutDiffusion is sampled with DPM-Solver [22]. The hyperparameters of LayoutDiffusion and LayoutDiffusion-small are listed in Tab. 4

D. Evaluation

D.1. Datasets

COCO-Stuff [5]. COCO [21] is a large-scale object detection, segmentation, and captioning dataset. COCO-Stuff [5] augments all 164K images of the COCO [21] dataset with pixel-level stuff annotations. These annotations can be used for scene understanding tasks like semantic segmentation, object detection and image captioning. COCO-Stuff [5] contains 80 categories of thing and 91 categories of stuff, respectively. Following the settings of LostGAN-v2 [41], we use the COCO 2017 Stuff Segmentation Challenge subset containing 40K / 5k / 5k images for train / val / test-dev set. Segmentation annotation is not used. We use images in the train and val set with 3 to 8 objects that cover more than 2% of the image and not belong to ‘crowd’. Finally, there are 25,210 train and 3,097 val images. Some previous works [2, 14] don’t filter objects belong to ‘crowd’, causing different numbers of images. Specifically, there are 24,972 train and 3,074 val images. LAMA [20] uses the full COCO-Stuff [5] 2017 dataset, leaving 74,777 train and 3,097 val images. Tab. 7 summarizes the difference in the number of images by different filtering methods.

use	filter	train			val		
deprecated	‘crowd’	image	object	object/image	image	object	object/image
✓		74,121	411,682	5.55	3,074	17,100	5.56
		24,972	138,162	5.53	3,074	17,100	5.56
✓	✓	74,777	414,443	5.54	3,097	17,191	5.55
	✓	25,210	139,175	5.52	3,097	17,191	5.55

Table 7. The difference in the number of images by different filtering methods, **use deprecated** means use COCO 2017 Stuff Segmentation Challenge subset, **filter ‘crowd’** means filter objects belong to ‘crowd’.

Visual Genome [19]. Following the settings of Sg2Im [14], we experiment on Visual Genome [19] version 1.4 (VG) which comprises 108,077 images annotated with scene graphs. Visual Genome [19] collects images with dense annotations of objects, attributes, and relationships. Here, we only use bounding boxes. We divide the data into 80% / 10% / 10% for the train / val / test set. We select the object / relationship categories occurring at least 2000 / 500 times in the train set, respectively, and select the images with 3 to 30 bounding boxes and ignoring all small objects. Finally, the train / val / test set has 62,565 / 5,062 / 5,096 images.

D.2. Evaluation Metrics

Comprehensive evaluations of generated images remains a challenge. We use six metrics, from image-level to layout-level, to evaluate the quality of the generated images and the layout control from different aspects.

Fréchet Inception Distance (FID) [9] shows the difference between the real images and the generated images by using an ImageNet-pretrained Inception-V3 [42] network and computing the Fréchet distance between two Gaussian distributions

fitted to generated images and real images respectively. We save the GT images as real images when sampling the generated images. The real images and the generated images are saved to two folder respectively. Then compute the FID score, based on the official code of FID[†]. For FID, the lower the score, the smaller the difference between the generated images and the real images, meaning the generator model is better.

Inception Score (IS) [35] shows the overall quality of the generated images by using an Inception-V3 [42] network pretrained on the ImageNet-1000 classification benchmark and computing a score(statistics) of the network’s outputs with generated images of a generator model. IS measures the quality of images on two aspects: clarity and diversity. We only need the generated images to compute the IS score, based on the official code of IS[†]. For IS, the higher the score, the better the quality of generated images, meaning the generator model is better.

Diversity Score (DS) measures the diversity between the generated images from the same layout by comparing the perceptual similarity in a DNN feature space between them. Here, we adopt the LPIPS [49] metric. For each sample, we repeat two times, use the two images from the same layout to compute the DS score, then calculate mean and std of these scores as the reported DS score, based on the official code of DS[†]. For DS, the higher the score, the better the diversity between the generated images, meaning the generator model is better.

YOLO Score [20] uses a pretrained YOLOv4 [4] model to evaluate bbox mAP on 80 thing categories based on the official code of LAMA[†] and the official code of YOLOv4[†]. YOLO Score [20] is proposed to evaluate the alignment and fidelity of generated objects, measuring how generated objects are recognizable when even the layout is unknown. YOLO [4, 30] is a well-known series of object detector, inferring layouts from the given images. Before send images to detector, they are upsampled to 512×512. And different from LAMA [20], since we filter the objects and images in datasets, we think it is better to evaluate bbox mAP only on filtered annotations.

Classification Score (CAS) [29] measures classification accuracy of layout areas on generated images. We crop the GT box area of images and resize objects at a resolution of 32×32 with their class. Then train a ResNet101 [8] classifier with cropped images on generated images and test it on cropped images on real images, based on a widely used codebase of image classification[†]. For CAS, the higher the score, the better the quality of layout control, meaning the generator model is better.

[†]the official code of FID [9]:<https://github.com/bioinf-jku/TTUR>

[†]the official code of IS [35]:<https://github.com/openai/improved-gan>

[†]the official code of DS [49]:<https://github.com/richzhang/PerceptualSimilarity>

[†]the official code of LAMA [20]:<https://github.com/ZejianLi/LAMA>

[†]the official code of YOLOv4 [4]:<https://github.com/AlexeyAB/darknet>

[†]the widely used codebase of image classification: https://github.com/hysts/pytorch_image_classification

References

- [1] Martin Arjovsky and Léon Bottou. Towards principled methods for training generative adversarial networks. *arXiv preprint arXiv:1701.04862*, 2017.
- [2] Oron Ashual and Lior Wolf. Specifying object attributes and relations in interactive scene generation. In *The IEEE International Conference on Computer Vision (ICCV)*, October 2019.
- [3] Fan Bao, Chongxuan Li, Jun Zhu, and Bo Zhang. Analytic-dpm: an analytic estimate of the optimal reverse variance in diffusion probabilistic models. In *ICLR*, 2021.
- [4] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection, 2020.
- [5] Holger Caesar, Jasper Uijlings, and Vittorio Ferrari. Coco-stuff: Thing and stuff classes in context. In *CVPR*, pages 1209–1218, 2018.
- [6] Florinel-Alin Croitoru, Vlad Hondru, Radu Tudor Ionescu, and Mubarak Shah. Diffusion models in vision: A survey. *arXiv preprint arXiv:2209.04747*, 2022.
- [7] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [9] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in neural information processing systems*, pages 6626–6637, 2017.
- [10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- [11] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.
- [12] Seunghoon Hong, Dingdong Yang, Jongwook Choi, and Honglak Lee. Inferring semantic layout for hierarchical text-to-image synthesis. In *CVPR*, 2018.
- [13] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *CVPR*, 2017.
- [14] Justin Johnson, Agrim Gupta, and Li Fei-Fei. Image generation from scene graphs. In *CVPR*, pages 1219–1228, 2018.
- [15] Levent Karacan, Zeynep Akata, Aykut Erdem, and Erkut Erdem. Learning to generate images of outdoor scenes from attributes and semantic layouts. *arXiv:1612.00215*, 2016.
- [16] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4401–4410, 2019.
- [17] Bahjat Kawar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising diffusion restoration models. In *Advances in Neural Information Processing Systems*, 2022.
- [18] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9404–9413, 2019.
- [19] Ranjay Krishna, Yuke Zhu, Oliver Groth, Justin Johnson, Kenji Hata, Joshua Kravitz, Stephanie Chen, Yannis Kalantidis, Li-Jia Li, David A Shamma, et al. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *IJCV*, 123(1):32–73, 2017.
- [20] Zejian Li, Jingyu Wu, Immanuel Koh, Yongchuan Tang, and Lingyun Sun. Image synthesis from layout with locality-aware mask adaptation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 13819–13828, 2021.
- [21] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2014.
- [22] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *arXiv preprint arXiv:2206.00927*, 2022.
- [23] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11461–11471, 2022.
- [24] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.
- [25] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*, 2021.
- [26] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021.
- [27] Taesung Park, Ming-Yu Liu, Ting-Chun Wang, and Jun-Yan Zhu. Semantic image synthesis with spatially-adaptive normalization. In *CVPR*, 2019.

- [28] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.
- [29] Suman Ravuri and Oriol Vinyals. Classification accuracy score for conditional generative models. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [30] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [31] Scott E Reed, Zeynep Akata, Santosh Mohan, Samuel Tenka, Bernt Schiele, and Honglak Lee. Learning what and where to draw. In *NIPS*, 2016.
- [32] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, pages 10684–10695, 2022.
- [33] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S Sara Mahdavi, Rapha Gontijo Lopes, et al. Photorealistic text-to-image diffusion models with deep language understanding. *arXiv preprint arXiv:2205.11487*, 2022.
- [34] Chitwan Saharia, Jonathan Ho, William Chan, Tim Salimans, David J Fleet, and Mohammad Norouzi. Image super-resolution via iterative refinement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [35] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *NIPS*, 2016.
- [36] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. *arXiv:1503.03585*, 2015.
- [37] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv:2010.02502*, October 2020.
- [38] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *arXiv:arXiv:1907.05600*, 2020.
- [39] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2020.
- [40] Wei Sun and Tianfu Wu. Image synthesis from reconfigurable layout and style. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 10531–10540, 2019.
- [41] Wei Sun and Tianfu Wu. Learning layout and style reconfigurable gans for controllable image synthesis. *IEEE TPAMI*, 44(9):5070–5087, 2021.
- [42] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision, 2015.
- [43] Fuwen Tan, Song Feng, and Vicente Ordonez. Text2scene: generating abstract scenes from textual descriptions. *arXiv:1809.01110*, 2018.
- [44] Anwaar Ulhaq, Naveed Akhtar, and Ganna Pogrebna. Efficient diffusion models for vision: A survey. *arXiv preprint arXiv:2210.09292*, 2022.
- [45] Bo Wang, Tao Wu, Minfeng Zhu, and Peng Du. Interactive image synthesis with panoptic layout generation. In *CVPR*, pages 7783–7792, 2022.
- [46] Ting-Chun Wang, Ming-Yu Liu, Jun-Yan Zhu, Andrew Tao, Jan Kautz, and Bryan Catanzaro. High-resolution image synthesis and semantic manipulation with conditional gans. In *CVPR*, 2018.
- [47] Ling Yang, Zhilin Huang, Yang Song, Shenda Hong, Guohao Li, Wentao Zhang, Bin Cui, Bernard Ghanem, and Ming-Hsuan Yang. Diffusion-based scene graph to image generation with masked contrastive pre-training. *arXiv preprint arXiv:2211.11138*, 2022.
- [48] Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Yingxia Shao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. Diffusion models: A comprehensive survey of methods and applications. *arXiv preprint arXiv:2209.00796*, 2022.
- [49] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [50] Bo Zhao, Lili Meng, Weidong Yin, and Leonid Sigal. Image generation from layout. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8584–8593, 2019.
- [51] Guangcong Zheng, Shengming Li, Hui Wang, Taiping Yao, Yang Chen, Shouhong Ding, and Xi Li. Entropy-driven sampling and training scheme for conditional diffusion generation. In *European Conference on Computer Vision*, pages 754–769. Springer, 2022.